Actividad Python - Estructuras de Control y Listas

Guía de ejecución y funcionamiento

Cómo ejecutar los ejercicios

1. Instala Python 3. En Windows, durante la instalación conviene activar la opción para agregar Python al PATH.

2. Abre una terminal en la carpeta phyton/Estructuras_Control_y_Listas. Cada ejercicio es un programa independiente.

3. Comprueba la instalación con python --version; en algunos equipos Windows también puedes usar py --version.

4. Ejecuta un ejercicio, por ejemplo: python 01_if_numero_mayor_100.py. Escribe los datos cuando el programa los solicite.

También puedes abrir cualquier archivo en Visual Studio Code, IDLE u otro editor de Python y ejecutarlo desde allí.

Ejercicios incluidos

01. Estructuras condicionales: if Archivos 01-10	02. Estructuras condicionales: elif Archivos 11-20
03. Estructuras condicionales: else Archivos 21-30	04. Ciclo while Archivos 31-40
05. Ciclo for Archivos 41-50	06. Función range() Archivos 51-60
07. Control de flujo: break, continue y pass Archivos 61-70	08. Ejercicios integradores Archivos 71-72
09. Listas en Python Archivos 73-80	Total 80 programas Python

1. Número mayor a 100

Requerimiento de la actividad: Escriba un programa que lea un número entero y muestre un mensaje si este es mayor estrictamente a 100.

Archivo: 01_if_numero_mayor_100.py

```
numero = int(input("Ingrese un número entero: "))

if numero > 100:
    print("El número es mayor que 100.")
else:
    print("El número no es mayor que 100.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 01_if_numero_mayor_100.py

Datos de ejemplo: 101

Salida de referencia

```
Ingrese un número entero: El número es mayor que 100.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

2. Múltiplo de 5

Requerimiento de la actividad: Cree un script en VS Code que reciba un número y valide si es exactamente múltiplo de 5.

Archivo: 02_if_multiplo_de_5.py

```
numero = int(input("Ingrese un número entero: "))

if numero % 5 == 0:
    print(f"{numero} es múltiplo de 5.")
else:
    print(f"{numero} no es múltiplo de 5.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 02_if_multiplo_de_5.py

Datos de ejemplo: 20

Salida de referencia

```
Ingrese un número entero: 20 es múltiplo de 5.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

3. Bonificación laboral

Requerimiento de la actividad: Verifique si los años de servicio de un empleado superan los 5 años para otorgarle un bono.

Archivo: 03_if_bonificacion_laboral.py

```
anios_servicio = int(input("Ingrese los años de servicio: "))

if anios_servicio > 5:
    print("El empleado recibe la bonificación.")
else:
    print("El empleado todavía no cumple los años requeridos.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 03_if_bonificacion_laboral.py

Datos de ejemplo: 6

Salida de referencia

```
Ingrese los años de servicio: El empleado recibe la bonificación.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

4. Capacidad máxima de auditorio

Requerimiento de la actividad: Si el número de asistentes supera los 50, imprima alerta de aforo completo.

Archivo: 04_if_aforo_auditorio.py

```
asistentes = int(input("Ingrese la cantidad de asistentes: "))

if asistentes > 50:
    print("Alerta: se superó la capacidad máxima del auditorio.")
else:
    print("El aforo está dentro de la capacidad permitida.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 04_if_aforo_auditorio.py

Datos de ejemplo: 51

Salida de referencia

```
Ingrese la cantidad de asistentes: Alerta: se superó la capacidad máxima del auditorio.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

5. Calificación perfecta

Requerimiento de la actividad: Valide si la nota obtenida por un aprendiz es exactamente igual a 100.

Archivo: 05_if_calificacion_perfecta.py

```
nota = float(input("Ingrese la calificación del aprendiz: "))

if nota == 100:
    print("La calificación es perfecta.")
else:
    print("La calificación no es 100.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 05_if_calificacion_perfecta.py

Datos de ejemplo: 100

Salida de referencia

```
Ingrese la calificación del aprendiz: La calificación es perfecta.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

6. Descuento tercera edad

Requerimiento de la actividad: Verifique si la edad ingresada es igual o mayor a 60 años.

Archivo: 06_if_descuento_tercera_edad.py

```
edad = int(input("Ingrese la edad: "))

if edad >= 60:
    print("Aplica el descuento de tercera edad.")
else:
    print("No aplica el descuento de tercera edad.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 06_if_descuento_tercera_edad.py

Datos de ejemplo: 60

Salida de referencia

```
Ingrese la edad: Aplica el descuento de tercera edad.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

7. Longitud mínima de usuario

Requerimiento de la actividad: Compruebe si el nombre de usuario tiene más de 6 caracteres.

Archivo: 07_if_longitud_usuario.py

```
usuario = input("Ingrese un nombre de usuario: ")

if len(usuario) > 6:
    print("El nombre de usuario supera los 6 caracteres.")
else:
    print("El nombre de usuario debe tener más de 6 caracteres.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 07_if_longitud_usuario.py

Datos de ejemplo: usuario_largo

Salida de referencia

```
Ingrese un nombre de usuario: El nombre de usuario supera los 6 caracteres.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

8. Presupuesto familiar

Requerimiento de la actividad: Valide si los gastos mensuales superan el ingreso base del hogar.

Archivo: 08_if_presupuesto_familiar.py

```
ingreso = float(input("Ingrese el ingreso mensual del hogar: "))
gastos = float(input("Ingrese los gastos mensuales del hogar: "))

if gastos > ingreso:
    print("Alerta: los gastos superan el ingreso familiar.")
else:
    print("El presupuesto está equilibrado.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 08_if_presupuesto_familiar.py

Datos de ejemplo: 1000 | 1200

Salida de referencia

```
Ingrese el ingreso mensual del hogar: Ingrese los gastos mensuales del hogar: Alerta: los gastos superan el
ingreso familiar.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

9. Nivel de brillo automático

Requerimiento de la actividad: Si el sensor de luz ambiental arroja un valor menor a 20, active el modo nocturno.

Archivo: 09_if_brillo_automatico.py

```
nivel_luz = float(input("Ingrese el nivel de luz ambiental: "))

if nivel_luz < 20:
    print("Modo nocturno activado.")
else:
    print("Se mantiene el modo normal.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 09_if_brillo_automatico.py

Datos de ejemplo: 10

Salida de referencia

```
Ingrese el nivel de luz ambiental: Modo nocturno activado.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

10. Peso de equipaje permitido

Requerimiento de la actividad: Verifique si el peso de la maleta excede los 23 kilogramos reglamentarios.

Archivo: 10_if_peso_equipaje.py

```
peso_kg = float(input("Ingrese el peso de la maleta en kilogramos: "))

if peso_kg > 23:
    print("La maleta excede el límite de 23 kg.")
else:
    print("La maleta cumple con el límite de equipaje.")
```

Cómo funciona: El programa compara la entrada con la condición indicada y muestra el mensaje correspondiente al resultado.

Cómo ejecutar: python 10_if_peso_equipaje.py

Datos de ejemplo: 24

Salida de referencia

```
Ingrese el peso de la maleta en kilogramos: La maleta excede el límite de 23 kg.
```

Conceptos clave: input(), conversión de tipos, operadores de comparación, if y else.

11. Rango de edad vital

Requerimiento de la actividad: Clasifique en Niño, Adolescente, Adulto o Adulto Mayor según la edad ingresada.

Archivo: 11_elif_rango_edad.py

```
edad = int(input("Ingrese la edad: "))

if edad < 0:
    print("La edad no puede ser negativa.")
elif edad < 13:
    print("Clasificación: Niño.")
elif edad < 18:
    print("Clasificación: Adolescente.")
elif edad < 60:
    print("Clasificación: Adulto.")
else:
    print("Clasificación: Adulto Mayor.")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 11_elif_rango_edad.py

Datos de ejemplo: 20

Salida de referencia

```
Ingrese la edad: Clasificación: Adulto.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

12. Asignación de puntaje por medalla

Requerimiento de la actividad: Asigne 100 puntos por Oro, 50 por Plata y 20 por Bronce usando elif.

Archivo: 12_elif_puntaje_medalla.py

```
medalla = input("Ingrese la medalla obtenida (oro, plata o bronce): ").strip().lower()

if medalla == "oro":
    puntos = 100
elif medalla == "plata":
    puntos = 50
elif medalla == "bronce":
    puntos = 20
else:
    puntos = 0
    print("Medalla no reconocida.")

print(f"Puntaje asignado: {puntos}")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 12_elif_puntaje_medalla.py

Datos de ejemplo: oro

Salida de referencia

```
Ingrese la medalla obtenida (oro, plata o bronce): Puntaje asignado: 100
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

13. Menú de operaciones aritméticas

Requerimiento de la actividad: Seleccione opción 1 para sumar, 2 para restar, 3 para multiplicar y 4 para dividir.

Archivo: 13_elif_menu_operaciones.py

```
numero1 = float(input("Ingrese el primer número: "))
numero2 = float(input("Ingrese el segundo número: "))
opcion = input("Operación (1 sumar, 2 restar, 3 multiplicar, 4 dividir): ")

if opcion == "1":
    print(f"Resultado: {numero1 + numero2}")
elif opcion == "2":
    print(f"Resultado: {numero1 - numero2}")
elif opcion == "3":
    print(f"Resultado: {numero1 * numero2}")
elif opcion == "4":
    if numero2 == 0:
        print("No se puede dividir entre cero.")
    else:
        print(f"Resultado: {numero1 / numero2}")
else:
    print("Opción no válida.")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 13_elif_menu_operaciones.py

Datos de ejemplo: 8 | 2 | 1

Salida de referencia

```
Ingrese el primer número: Ingrese el segundo número: Operación (1 sumar, 2 restar, 3 multiplicar, 4 dividir):
Resultado: 10.0
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

14. Dificultad de juego

Requerimiento de la actividad: Establezca nivel Fácil, Medio, Difícil o Extremo según el selector numérico (1-4).

Archivo: 14_elif_dificultad_juego.py

```
nivel = int(input("Seleccione un nivel (1 a 4): "))

if nivel == 1:
    print("Dificultad: Fácil.")
elif nivel == 2:
    print("Dificultad: Medio.")
elif nivel == 3:
    print("Dificultad: Difícil.")
elif nivel == 4:
    print("Dificultad: Extremo.")
else:
    print("Seleccione un nivel entre 1 y 4.")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 14_elif_dificultad_juego.py

Datos de ejemplo: 4

Salida de referencia

```
Seleccione un nivel (1 a 4): Dificultad: Extremo.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

15. Descuento por categoría de cliente

Requerimiento de la actividad: Cliente VIP, Regular o Nuevo con sus respectivos porcentajes de rebaja.

Archivo: 15_elif_descuento_cliente.py

```
categoria = input("Categoría de cliente (VIP, regular o nuevo): ").strip().lower()
compra = float(input("Ingrese el valor de la compra: "))

if categoria == "vip":
    descuento = 0.20
elif categoria == "regular":
    descuento = 0.10
elif categoria == "nuevo":
    descuento = 0.05
else:
    descuento = 0
    print("Categoría no reconocida; no se aplica descuento.")

total = compra * (1 - descuento)
print(f"Descuento: {descuento:.0%} | Total a pagar: ${total:,.2f}")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 15_elif_descuento_cliente.py

Datos de ejemplo: vip | 100

Salida de referencia

```
Categoría de cliente (VIP, regular o nuevo): Ingrese el valor de la compra: Descuento: 20% | Total a pagar: $80.
00
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

16. Calificación de calidad de aire (AQI)

Requerimiento de la actividad: Bueno, Moderado, Dañino para grupos sensibles o Peligroso.

Archivo: 16_elif_calidad_aire_aqi.py

```
aqi = int(input("Ingrese el índice de calidad del aire (AQI): "))

if aqi < 0:
    print("El AQI no puede ser negativo.")
elif aqi <= 50:
    print("Calidad del aire: Bueno.")
elif aqi <= 100:
    print("Calidad del aire: Moderado.")
elif aqi <= 150:
    print("Calidad del aire: Dañino para grupos sensibles.")
else:
    print("Calidad del aire: Peligroso.")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 16_elif_calidad_aire_aqi.py

Datos de ejemplo: 75

Salida de referencia

```
Ingrese el índice de calidad del aire (AQI): Calidad del aire: Moderado.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

17. Tipo de vehículo por número de ruedas

Requerimiento de la actividad: Moto (2), Auto (4), Bus (6).

Archivo: 17_elif_tipo_vehiculo_ruedas.py

```
ruedas = int(input("Ingrese el número de ruedas del vehículo: "))

if ruedas == 2:
    print("Tipo de vehículo: Moto.")
elif ruedas == 4:
    print("Tipo de vehículo: Auto.")
elif ruedas == 6:
    print("Tipo de vehículo: Bus.")
else:
    print("Tipo de vehículo no identificado.")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 17_elif_tipo_vehiculo_ruedas.py

Datos de ejemplo: 4

Salida de referencia

```
Ingrese el número de ruedas del vehículo: Tipo de vehículo: Auto.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

18. Turno de trabajo laboral

Requerimiento de la actividad: Mañana, Tarde o Noche según la hora de ingreso.

Archivo: 18_elif_turno_trabajo.py

```
hora = int(input("Ingrese la hora de ingreso (0 a 23): "))

if not 0 <= hora <= 23:
    print("La hora debe estar entre 0 y 23.")
elif hora < 12:
    print("Turno: Mañana.")
elif hora < 18:
    print("Turno: Tarde.")
else:
    print("Turno: Noche.")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 18_elif_turno_trabajo.py

Datos de ejemplo: 14

Salida de referencia

```
Ingrese la hora de ingreso (0 a 23): Turno: Tarde.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

19. Tamaño de archivo digital

Requerimiento de la actividad: KB, MB o GB según el valor numérico de bytes.

Archivo: 19_elif_tamano_archivo.py

```
bytes_archivo = float(input("Ingrese el tamaño del archivo en bytes: "))

if bytes_archivo < 0:
    print("El tamaño no puede ser negativo.")
elif bytes_archivo < 1024:
    print(f"Tamaño: {bytes_archivo:.2f} bytes")
elif bytes_archivo < 1024**2:
    print(f"Tamaño: {bytes_archivo / 1024:.2f} KB")
elif bytes_archivo < 1024**3:
    print(f"Tamaño: {bytes_archivo / 1024**2:.2f} MB")
else:
    print(f"Tamaño: {bytes_archivo / 1024**3:.2f} GB")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 19_elif_tamano_archivo.py

Datos de ejemplo: 2048

Salida de referencia

```
Ingrese el tamaño del archivo en bytes: Tamaño: 2.00 KB
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

20. Clasificación de huracanes Saffir-Simpson

Requerimiento de la actividad: Categorías del 1 al 5 según la velocidad del viento.

Archivo: 20_elif_categoria_huracan.py

```
viento_kmh = float(input("Ingrese la velocidad del viento en km/h: "))

if viento_kmh < 0:
    print("La velocidad no puede ser negativa.")
elif viento_kmh < 119:
    print("No alcanza la categoría de huracán.")
elif viento_kmh <= 153:
    print("Huracán categoría 1.")
elif viento_kmh <= 177:
    print("Huracán categoría 2.")
elif viento_kmh <= 208:
    print("Huracán categoría 3.")
elif viento_kmh <= 251:
    print("Huracán categoría 4.")
else:
    print("Huracán categoría 5.")
```

Cómo funciona: El programa evalúa las alternativas en orden y selecciona la categoría u operación que corresponde a la entrada.

Cómo ejecutar: python 20_elif_categoria_huracan.py

Datos de ejemplo: 180

Salida de referencia

```
Ingrese la velocidad del viento en km/h: Huracán categoría 3.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

21. Aprobación de crédito bancario

Requerimiento de la actividad: Si el sueldo supera 1.5 millones se aprueba, de lo contrario se rechaza.

Archivo: 21_else_credito_bancario.py

```
sueldo = float(input("Ingrese el sueldo mensual: "))

if sueldo > 1_500_000:
    print("Crédito aprobado.")
else:
    print("Crédito rechazado.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 21_else_credito_bancario.py

Datos de ejemplo: 1600000

Salida de referencia

```
Ingrese el sueldo mensual: Crédito aprobado.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

22. Control de temperatura de motor

Requerimiento de la actividad: Normal o Apagado de emergencia por sobrecalentamiento.

Archivo: 22_else_temperatura_motor.py

```
temperatura = float(input("Ingrese la temperatura del motor en °C: "))
limite_seguro = 90

if temperatura <= limite_seguro:
    print("Temperatura normal.")
else:
    print("Sobrecalentamiento: apagado de emergencia.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 22_else_temperatura_motor.py

Datos de ejemplo: 100

Salida de referencia

```
Ingrese la temperatura del motor en °C: Sobrecalentamiento: apagado de emergencia.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

23. Verificación de año par o impar

Requerimiento de la actividad: Determine si un año ingresado es par o impar.

Archivo: 23_else_paridad_anio.py

```
anio = int(input("Ingrese un año: "))

if anio % 2 == 0:
    print(f"El año {anio} es par.")
else:
    print(f"El año {anio} es impar.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 23_else_paridad_anio.py

Datos de ejemplo: 2025

Salida de referencia

```
Ingrese un año: El año 2025 es impar.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

24. Estado de beca estudiantil

Requerimiento de la actividad: Si el promedio es >= 4.5 se otorga beca, de lo contrario paga tarifa estándar.

Archivo: 24_else_beca_estudiantil.py

```
promedio = float(input("Ingrese el promedio del estudiante: "))

if promedio >= 4.5:
    print("Se otorga la beca estudiantil.")
else:
    print("Se paga la tarifa estándar.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 24_else_beca_estudiantil.py

Datos de ejemplo: 4.7

Salida de referencia

```
Ingrese el promedio del estudiante: Se otorga la beca estudiantil.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

25. Permiso de conducir nocturno

Requerimiento de la actividad: Valide si la hora permite conducir sin restricciones o requiere autorización.

Archivo: 25_else_permiso_conducir_nocturno.py

```
hora = int(input("Ingrese la hora (0 a 23): "))

if not 0 <= hora <= 23:
    print("La hora debe estar entre 0 y 23.")
elif 6 <= hora < 23:
    print("Puede conducir sin restricciones horarias.")
else:
    print("Se requiere autorización para conducir a esta hora.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 25_else_permiso_conducir_nocturno.py

Datos de ejemplo: 2

Salida de referencia

```
Ingrese la hora (0 a 23): Se requiere autorización para conducir a esta hora.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

26. Verificación de stock mínimo

Requerimiento de la actividad: Aviso de reabastecimiento o stock suficiente.

Archivo: 26_else_stock_minimo.py

```
stock = int(input("Ingrese la cantidad disponible: "))
stock_minimo = 10

if stock >= stock_minimo:
    print("Stock suficiente.")
else:
    print("Aviso: reabastecer inventario.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 26_else_stock_minimo.py

Datos de ejemplo: 4

Salida de referencia

```
Ingrese la cantidad disponible: Aviso: reabastecer inventario.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

27. Acceso a sección VIP

Requerimiento de la actividad: Membresía activa o entrada general.

Archivo: 27_else_acceso_vip.py

```
membresia_activa = input("¿Tiene una membresía VIP activa? (sí/no): ").strip().lower()

if membresia_activa in ("si", "sí"):
    print("Acceso a la sección VIP.")
else:
    print("Acceso a la entrada general.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 27_else_acceso_vip.py

Datos de ejemplo: sí

Salida de referencia

```
¿Tiene una membresía VIP activa? (sí/no): Acceso a la sección VIP.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

28. Paridad de número de cédula

Requerimiento de la actividad: Valide pico y cédula par o impar.

Archivo: 28_else_paridad_cedula.py

```
numero_cedula = int(input("Ingrese el último dígito de la cédula: "))

if numero_cedula % 2 == 0:
    print("Turno de pico y cédula para números pares.")
else:
    print("Turno de pico y cédula para números impares.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 28_else_paridad_cedula.py

Datos de ejemplo: 7

Salida de referencia

```
Ingrese el último dígito de la cédula: Turno de pico y cédula para números impares.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

29. Mayor o menor entre dos valores

Requerimiento de la actividad: Compare dos números e indique cuál es mayor o si son iguales con else.

Archivo: 29_else_comparar_valores.py

```
valor1 = float(input("Ingrese el primer valor: "))
valor2 = float(input("Ingrese el segundo valor: "))

if valor1 > valor2:
    print(f"El mayor es {valor1}.")
elif valor2 > valor1:
    print(f"El mayor es {valor2}.")
else:
    print("Los dos valores son iguales.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 29_else_comparar_valores.py

Datos de ejemplo: 8 | 3

Salida de referencia

```
Ingrese el primer valor: Ingrese el segundo valor: El mayor es 8.0.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

30. Validación de formato de correo

Requerimiento de la actividad: Si contiene '@' es válido, de lo contrario inválido.

Archivo: 30_else_validar_correo.py

```
correo = input("Ingrese una dirección de correo electrónico: ").strip()

if "@" in correo:
    print("El correo contiene el carácter @.")
else:
    print("El correo no contiene el carácter @.")
```

Cómo funciona: La condición principal selecciona una respuesta; else presenta el resultado alternativo cuando no se cumple.

Cómo ejecutar: python 30_else_validar_correo.py

Datos de ejemplo: ana@example.com

Salida de referencia

```
Ingrese una dirección de correo electrónico: El correo contiene el carácter @.
```

Conceptos clave: input(), if, elif, else y operadores de comparación.

31. Imprimir números del 1 al 20

Requerimiento de la actividad: Use un ciclo while para mostrar los números enteros del 1 al 20.

Archivo: 31_while_numeros_1_a_20.py

```
numero = 1

while numero <= 20:
    print(numero)
    numero += 1
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 31_while_numeros_1_a_20.py

Salida de referencia

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

32. Sumar números ingresados hasta cero

Requerimiento de la actividad: Sume números enteros ingresados repetidamente hasta que el usuario digite 0.

Archivo: 32_while_suma_hasta_cero.py

```
suma = 0

while True:
    numero = int(input("Ingrese un número (0 para terminar): "))
    if numero == 0:
        break
    suma += numero

print(f"Suma total: {suma}")
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 32_while_suma_hasta_cero.py

Datos de ejemplo: 5 | -2 | 0

Salida de referencia

```
Ingrese un número (0 para terminar): Ingrese un número (0 para terminar): Ingrese un número (0 para terminar):
Suma total: 3
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

33. Tabla de multiplicar personalizada

Requerimiento de la actividad: Genere la tabla de multiplicar de un número ingresado del 1 al 10.

Archivo: 33_while_tabla_multiplicar.py

```
numero = int(input("Ingrese un número para generar su tabla: "))
multiplicador = 1

while multiplicador <= 10:
    print(f"{numero} x {multiplicador} = {numero * multiplicador}")
    multiplicador += 1
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 33_while_tabla_multiplicar.py

Datos de ejemplo: 7

Salida de referencia

```
Ingrese un número para generar su tabla: 7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

34. Adivinar número secreto

Requerimiento de la actividad: Ciclo que permita adivinar un número fijo hasta acertar.

Archivo: 34_while_adivinar_numero.py

```
numero_secreto = 7
intento = None

while intento != numero_secreto:
    intento = int(input("Adivine el número secreto (entre 1 y 10): "))
    if intento < numero_secreto:
        print("El número secreto es mayor.")
    elif intento > numero_secreto:
        print("El número secreto es menor.")

print("¡Adivinaste!")
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 34_while_adivinar_numero.py

Datos de ejemplo: 3 | 7

Salida de referencia

```
Adivine el número secreto (entre 1 y 10): El número secreto es mayor.
Adivine el número secreto (entre 1 y 10): ¡Adivinaste!
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

35. Menú interactivo repetitivo

Requerimiento de la actividad: Muestre un menú de opciones que se repita hasta que se elija salir (opción 5).

Archivo: 35_while_menu_interactivo.py

```
while True:
    print("\n1. Saludar\n2. Mostrar una frase\n3. Mostrar un número\n4. Ayuda\n5. Salir")
    opcion = input("Seleccione una opción: ")

    if opcion == "1":
        print("¡Hola!")
    elif opcion == "2":
        print("Python permite resolver problemas paso a paso.")
    elif opcion == "3":
        print("El número seleccionado es 5.")
    elif opcion == "4":
        print("Elija una opción del 1 al 5.")
    elif opcion == "5":
        print("Fin del programa.")
        break
    else:
        print("Opción no válida.")
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 35_while_menu_interactivo.py

Datos de ejemplo: 5

Salida de referencia

```
1. Saludar
2. Mostrar una frase
3. Mostrar un número
4. Ayuda
5. Salir
Seleccione una opción: Fin del programa.
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

36. Divisiones sucesivas por 2

Requerimiento de la actividad: Divida un número entre 2 sucesivamente hasta que sea menor a 1.

Archivo: 36_while_divisiones_por_dos.py

```
numero = float(input("Ingrese un número mayor o igual a 1: "))

if numero <= 0:
    print("Ingrese un número positivo.")
else:
    while numero >= 1:
        print(f"{numero:g}")
        numero /= 2
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 36_while_divisiones_por_dos.py

Datos de ejemplo: 8

Salida de referencia

```
Ingrese un número mayor o igual a 1: 8
4
2
1
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

37. Control de intentos de cajero

Requerimiento de la actividad: Permita máximo 3 intentos de PIN bancario.

Archivo: 37_while_intentos_cajero.py

```
pin_correcto = "2468"
intentos = 0

while intentos < 3:
    pin = input("Ingrese su PIN: ")
    intentos += 1
    if pin == pin_correcto:
        print("Acceso autorizado.")
        break
    print(f"PIN incorrecto. Intentos restantes: {3 - intentos}")
else:
    print("Tarjeta bloqueada por superar los 3 intentos.")
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 37_while_intentos_cajero.py

Datos de ejemplo: 0000 | 0000 | 2468

Salida de referencia

```
Ingrese su PIN: PIN incorrecto. Intentos restantes: 2
Ingrese su PIN: PIN incorrecto. Intentos restantes: 1
Ingrese su PIN: Acceso autorizado.
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

38. Secuencia de Fibonacci con while

Requerimiento de la actividad: Genere los primeros 10 términos de la serie.

Archivo: 38_while_fibonacci_10.py

```
anterior = 0
actual = 1
terminos = 0

while terminos < 10:
    print(anterior)
    anterior, actual = actual, anterior + actual
    terminos += 1
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 38_while_fibonacci_10.py

Salida de referencia

```
0
1
1
2
3
5
8
13
21
34
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

39. Impresión de caracteres de un texto

Requerimiento de la actividad: Recorra un string letra por letra usando índice y while.

Archivo: 39_while_caracteres_texto.py

```
texto = input("Ingrese un texto: ")
indice = 0

while indice < len(texto):
    print(texto[indice])
    indice += 1
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 39_while_caracteres_texto.py

Datos de ejemplo: hola

Salida de referencia

```
Ingrese un texto: h
o
l
a
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

40. Llenado de tanque de agua virtual

Requerimiento de la actividad: Incremente nivel de 10 en 10 hasta llegar a 100.

Archivo: 40_while_llenado_tanque.py

```
nivel = 0

while nivel < 100:
    nivel += 10
    print(f"Nivel del tanque: {nivel}%")

print("Tanque lleno.")
```

Cómo funciona: while repite las instrucciones mientras se mantenga la condición y actualiza los valores para llegar al final del ciclo.

Cómo ejecutar: python 40_while_llenado_tanque.py

Salida de referencia

```
Nivel del tanque: 10%
Nivel del tanque: 20%
Nivel del tanque: 30%
Nivel del tanque: 40%
Nivel del tanque: 50%
Nivel del tanque: 60%
Nivel del tanque: 70%
Nivel del tanque: 80%
Nivel del tanque: 90%
Nivel del tanque: 100%
Tanque lleno.
```

Conceptos clave: while, condiciones, contadores, acumuladores y break.

41. Recorrer nombres de tecnologías

Requerimiento de la actividad: Cree una lista con Python, Java, PHP y recórralos con un ciclo for.

Archivo: 41_for_nombres_tecnologias.py

```
tecnologias = ["Python", "Java", "PHP"]

for tecnologia in tecnologias:
    print(tecnologia)
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 41_for_nombres_tecnologias.py

Salida de referencia

```
Python
Java
PHP
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

42. Sumar salarios de una lista

Requerimiento de la actividad: Calcule la sumatoria total de 5 valores salariales almacenados en una lista.

Archivo: 42_for_sumar_salarios.py

```
salarios = []

for indice in range(5):
    salario = float(input(f"Ingrese el salario {indice + 1}: "))
    salarios.append(salario)

total = 0
for salario in salarios:
    total += salario

print(f"Sumatoria de salarios: ${total:,.2f}")
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 42_for_sumar_salarios.py

Datos de ejemplo: 100 | 200 | 300 | 400 | 500

Salida de referencia

```
Ingrese el salario 1: Ingrese el salario 2: Ingrese el salario 3: Ingrese el salario 4: Ingrese el salario 5:
Sumatoria de salarios: $1,500.00
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

43. Encontrar palabra más larga

Requerimiento de la actividad: Recorra una lista de palabras e imprima la de mayor longitud.

Archivo: 43_for_palabra_mas_larga.py

```
palabras = input("Ingrese palabras separadas por coma: ").split(",")
palabra_mas_larga = ""

for palabra in palabras:
    palabra = palabra.strip()
    if len(palabra) > len(palabra_mas_larga):
        palabra_mas_larga = palabra

if palabra_mas_larga:
    print(f"La palabra más larga es: {palabra_mas_larga}")
else:
    print("No se ingresaron palabras.")
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 43_for_palabra_mas_larga.py

Datos de ejemplo: sol,computadora,luz

Salida de referencia

```
Ingrese palabras separadas por coma: La palabra más larga es: computadora
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

44. Duplicar elementos de una lista

Requerimiento de la actividad: Multiplique por 2 cada número de una lista dada.

Archivo: 44_for_duplicar_lista.py

```
numeros = [3, 7, 10, 12]

for numero in numeros:
    print(f"{numero} duplicado es {numero * 2}")
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 44_for_duplicar_lista.py

Salida de referencia

```
3 duplicado es 6
7 duplicado es 14
10 duplicado es 20
12 duplicado es 24
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

45. Contar elementos mayores a 50

Requerimiento de la actividad: Dada una lista de notas, cuente cuántas son mayores a 50.

Archivo: 45_for_contar_notas_mayores_50.py

```
notas = [45, 72, 88, 50, 61]
cantidad_mayores_50 = 0

for nota in notas:
    if nota > 50:
        cantidad_mayores_50 += 1

print(f"Cantidad de notas mayores a 50: {cantidad_mayores_50}")
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 45_for_contar_notas_mayores_50.py

Salida de referencia

```
Cantidad de notas mayores a 50: 3
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

46. Generar correos institucionales

Requerimiento de la actividad: Concatene '@sena.edu.co' a una lista de nombres de usuario.

Archivo: 46_for_correos_institucionales.py

```
usuarios = input("Ingrese nombres de usuario separados por coma: ").split(",")

for usuario in usuarios:
    usuario = usuario.strip()
    if usuario:
        print(f"{usuario}@sena.edu.co")
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 46_for_correos_institucionales.py

Datos de ejemplo: ana,luis

Salida de referencia

```
Ingrese nombres de usuario separados por coma: ana@sena.edu.co
luis@sena.edu.co
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

47. Imprimir reverso de palabra con for

Requerimiento de la actividad: Recorra caracteres y procese inversión.

Archivo: 47_for_reverso_palabra.py

```
palabra = input("Ingrese una palabra: ")
reverso = ""

for caracter in palabra:
    reverso = caracter + reverso

print(f"Palabra invertida: {reverso}")
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 47_for_reverso_palabra.py

Datos de ejemplo: python

Salida de referencia

```
Ingrese una palabra: Palabra invertida: nohtyp
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

48. Validar stock mínimo en lista de tuplas

Requerimiento de la actividad: Revise inventario de productos y cantidades.

Archivo: 48_for_stock_lista_tuplas.py

```
inventario = [("Teclado", 12), ("Mouse", 4), ("Monitor", 8)]
stock_minimo = 5

for producto, cantidad in inventario:
    if cantidad < stock_minimo:
        print(f"{producto}: reabastecer (stock {cantidad}).")
    else:
        print(f"{producto}: stock suficiente ({cantidad}).")
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 48_for_stock_lista_tuplas.py

Salida de referencia

```
Teclado: stock suficiente (12).
Mouse: reabastecer (stock 4).
Monitor: stock suficiente (8).
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

49. Separar números positivos y negativos

Requerimiento de la actividad: Clasifique elementos de una lista numérica.

Archivo: 49_for_separar_positivos_negativos.py

```
numeros = [12, -4, 0, 7, -9, 3]
positivos = []
negativos = []

for numero in numeros:
    if numero > 0:
        positivos.append(numero)
    elif numero < 0:
        negativos.append(numero)

print(f"Positivos: {positivos}")
print(f"Negativos: {negativos}")
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 49_for_separar_positivos_negativos.py

Salida de referencia

```
Positivos: [12, 7, 3]
Negativos: [-4, -9]
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

50. Concatenar frase completa

Requerimiento de la actividad: Una una lista de palabras en una sola oración usando for.

Archivo: 50_for_concatenar_frase.py

```
palabras = input("Ingrese las palabras de una frase separadas por coma: ").split(",")
frase = ""

for palabra in palabras:
    palabra = palabra.strip()
    if palabra:
        if frase:
            frase += " "
        frase += palabra

print(frase)
```

Cómo funciona: for recorre cada elemento de la secuencia y realiza con él la operación solicitada.

Cómo ejecutar: python 50_for_concatenar_frase.py

Datos de ejemplo: hola,mundo

Salida de referencia

```
Ingrese las palabras de una frase separadas por coma: hola mundo
```

Conceptos clave: for, listas, secuencias, condiciones y acumuladores.

51. Números del 10 al 20

Requerimiento de la actividad: Use range para imprimir del 10 al 20.

Archivo: 51_range_numeros_10_a_20.py

```
for numero in range(10, 21):
    print(numero)
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 51_range_numeros_10_a_20.py

Salida de referencia

```
10
11
12
13
14
15
16
17
18
19
20
```

Conceptos clave: range(), inicio, límite, paso y for.

52. Números impares del 1 al 19

Requerimiento de la actividad: Genere impares usando el tercer parámetro de range.

Archivo: 52_range_impares_1_a_19.py

```
for numero in range(1, 20, 2):
    print(numero)
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 52_range_impares_1_a_19.py

Salida de referencia

```
1
3
5
7
9
11
13
15
17
19
```

Conceptos clave: range(), inicio, límite, paso y for.

53. Cuenta regresiva desde 10

Requerimiento de la actividad: Imprima del 10 al 1 en orden descendente.

Archivo: 53_range_cuenta_regresiva.py

```
for numero in range(10, 0, -1):
    print(numero)
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 53_range_cuenta_regresiva.py

Salida de referencia

```
10
9
8
7
6
5
4
3
2
1
```

Conceptos clave: range(), inicio, límite, paso y for.

54. Sumar pares entre 1 y 50

Requerimiento de la actividad: Acumule la suma de todos los pares en ese rango.

Archivo: 54_range_sumar_pares_1_a_50.py

```
suma = 0

for numero in range(2, 51, 2):
    suma += numero

print(f"Suma de los pares entre 1 y 50: {suma}")
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 54_range_sumar_pares_1_a_50.py

Salida de referencia

```
Suma de los pares entre 1 y 50: 650
```

Conceptos clave: range(), inicio, límite, paso y for.

55. Tabla del 9 completa

Requerimiento de la actividad: Multiplicaciones del 1 al 10 para el 9.

Archivo: 55_range_tabla_del_9.py

```
for multiplicador in range(1, 11):
    print(f"9 x {multiplicador} = {9 * multiplicador}")
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 55_range_tabla_del_9.py

Salida de referencia

```
9 x 1 = 9
9 x 2 = 18
9 x 3 = 27
9 x 4 = 36
9 x 5 = 45
9 x 6 = 54
9 x 7 = 63
9 x 8 = 72
9 x 9 = 81
9 x 10 = 90
```

Conceptos clave: range(), inicio, límite, paso y for.

56. Múltiplos de 4 hasta 40

Requerimiento de la actividad: Muestre los múltiplos utilizando saltos en range.

Archivo: 56_range_multiplos_de_4.py

```
for numero in range(4, 41, 4):
    print(numero)
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 56_range_multiplos_de_4.py

Salida de referencia

```
4
8
12
16
20
24
28
32
36
40
```

Conceptos clave: range(), inicio, límite, paso y for.

57. Cuadrados de 1 a 10

Requerimiento de la actividad: Calcule cuadrados combinando range y for.

Archivo: 57_range_cuadrados_1_a_10.py

```
for numero in range(1, 11):
    print(f"{numero} al cuadrado es {numero ** 2}")
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 57_range_cuadrados_1_a_10.py

Salida de referencia

```
1 al cuadrado es 1
2 al cuadrado es 4
3 al cuadrado es 9
4 al cuadrado es 16
5 al cuadrado es 25
6 al cuadrado es 36
7 al cuadrado es 49
8 al cuadrado es 64
9 al cuadrado es 81
10 al cuadrado es 100
```

Conceptos clave: range(), inicio, límite, paso y for.

58. Generar listado numérico dinámico

Requerimiento de la actividad: Rango basado en variables configurables.

Archivo: 58_range_dinamico.py

```
inicio = int(input("Ingrese el inicio del rango: "))
fin = int(input("Ingrese el límite final (no incluido): "))
paso = int(input("Ingrese el paso: "))

if paso == 0:
    print("El paso no puede ser cero.")
else:
    for numero in range(inicio, fin, paso):
        print(numero)
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 58_range_dinamico.py

Datos de ejemplo: 1 | 5 | 1

Salida de referencia

```
Ingrese el inicio del rango: Ingrese el límite final (no incluido): Ingrese el paso: 1
2
3
4
```

Conceptos clave: range(), inicio, límite, paso y for.

59. Secuencia descendente de 2 en 2

Requerimiento de la actividad: Desde 20 hasta 0 con paso -2.

Archivo: 59_range_descendente_de_2.py

```
for numero in range(20, -1, -2):
    print(numero)
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 59_range_descendente_de_2.py

Salida de referencia

```
20
18
16
14
12
10
8
6
4
2
0
```

Conceptos clave: range(), inicio, límite, paso y for.

60. Validar repetición de tareas

Requerimiento de la actividad: Ejecutar bloque 5 veces exactas.

Archivo: 60_range_repetir_tarea.py

```
for repeticion in range(1, 6):
    print(f"Ejecución {repeticion}: tarea realizada.")
```

Cómo funciona: range() genera la secuencia de enteros indicada y for procesa cada uno de sus valores.

Cómo ejecutar: python 60_range_repetir_tarea.py

Salida de referencia

```
Ejecución 1: tarea realizada.
Ejecución 2: tarea realizada.
Ejecución 3: tarea realizada.
Ejecución 4: tarea realizada.
Ejecución 5: tarea realizada.
```

Conceptos clave: range(), inicio, límite, paso y for.

61. Detener bucle al hallar número 7

Requerimiento de la actividad: Use break al encontrar el 7 en una lista.

Archivo: 61_control_break_hallar_7.py

```
numeros = [2, 4, 6, 7, 9, 11]

for numero in numeros:
    if numero == 7:
        print("Se encontró el número 7.")
        break
    print(f"Revisando: {numero}")
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 61_control_break_hallar_7.py

Salida de referencia

```
Revisando: 2
Revisando: 4
Revisando: 6
Se encontró el número 7.
```

Conceptos clave: break, continue, pass, for y while.

62. Saltar múltiplos de 3 con continue

Requerimiento de la actividad: Imprima del 1 al 10 omitiendo múltiplos de 3.

Archivo: 62_control_continue_multiplos_3.py

```
for numero in range(1, 11):
    if numero % 3 == 0:
        continue
    print(numero)
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 62_control_continue_multiplos_3.py

Salida de referencia

```
1
2
4
5
7
8
10
```

Conceptos clave: break, continue, pass, for y while.

63. Función en desarrollo con pass

Requerimiento de la actividad: Implemente un esqueleto de clase usando pass.

Archivo: 63_control_clase_con_pass.py

```
class ActividadEnDesarrollo:
    pass


print("La clase está definida y lista para completarse.")
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 63_control_clase_con_pass.py

Salida de referencia

```
La clase está definida y lista para completarse.
```

Conceptos clave: break, continue, pass, for y while.

64. Salir de login tras acierto

Requerimiento de la actividad: Detenga ciclo de intentos con break al acertar.

Archivo: 64_control_login_break.py

```
contrasena_correcta = "SENA2026"

for intento in range(1, 4):
    contrasena = input(f"Intento {intento} de 3. Ingrese la contraseña: ")
    if contrasena == contrasena_correcta:
        print("Acceso concedido.")
        break
    print("Contraseña incorrecta.")
else:
    print("Se agotaron los intentos.")
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 64_control_login_break.py

Datos de ejemplo: no | no | SENA2026

Salida de referencia

```
Intento 1 de 3. Ingrese la contraseña: Contraseña incorrecta.
Intento 2 de 3. Ingrese la contraseña: Contraseña incorrecta.
Intento 3 de 3. Ingrese la contraseña: Acceso concedido.
```

Conceptos clave: break, continue, pass, for y while.

65. Omitir vocales en frase

Requerimiento de la actividad: Imprima solo consonantes usando continue.

Archivo: 65_control_omitir_vocales.py

```
frase = input("Ingrese una frase: ")
vocales = "aeiouáéíóúAEIOUÁÉÍÓÚ"

for caracter in frase:
    if caracter in vocales:
        continue
    print(caracter, end="")

print()
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 65_control_omitir_vocales.py

Datos de ejemplo: Hola

Salida de referencia

```
Ingrese una frase: Hl
```

Conceptos clave: break, continue, pass, for y while.

66. Romper ciclo por límite de saldo

Requerimiento de la actividad: Detenga transacciones al quedar en cero.

Archivo: 66_control_limite_saldo.py

```
saldo = float(input("Ingrese el saldo inicial: "))

while saldo > 0:
    monto = float(input("Ingrese el monto de la transacción: "))
    if monto <= 0:
        print("El monto debe ser mayor que cero.")
        continue
    if monto > saldo:
        print(f"Transacción de ${monto:.2f} rechazada: saldo insuficiente.")
        continue
    saldo -= monto
    print(f"Se descontaron ${monto:.2f}. Saldo: ${saldo:.2f}")
    if saldo == 0:
        print("Saldo agotado; se detienen las transacciones.")
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 66_control_limite_saldo.py

Datos de ejemplo: 100 | 20 | 35 | 45

Salida de referencia

```
Ingrese el saldo inicial: Ingrese el monto de la transacción: Se descontaron $20.00. Saldo: $80.00
Ingrese el monto de la transacción: Se descontaron $35.00. Saldo: $45.00
Ingrese el monto de la transacción: Se descontaron $45.00. Saldo: $0.00
Saldo agotado; se detienen las transacciones.
```

Conceptos clave: break, continue, pass, for y while.

67. Validar rango con break

Requerimiento de la actividad: Finalice captura al recibir número válido.

Archivo: 67_control_validar_rango_break.py

```
while True:
    numero = int(input("Ingrese un número entre 1 y 10: "))
    if 1 <= numero <= 10:
        break
    print("Valor fuera de rango. Intente nuevamente.")

print(f"Número válido: {numero}")
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 67_control_validar_rango_break.py

Datos de ejemplo: 20 | 5

Salida de referencia

```
Ingrese un número entre 1 y 10: Valor fuera de rango. Intente nuevamente.
Ingrese un número entre 1 y 10: Número válido: 5
```

Conceptos clave: break, continue, pass, for y while.

68. Filtrar negativos con continue

Requerimiento de la actividad: Sume solo positivos saltando negativos.

Archivo: 68_control_sumar_positivos.py

```
numeros = [12, -5, 8, -2, 15]
suma = 0

for numero in numeros:
    if numero < 0:
        continue
    suma += numero

print(f"Suma de los números no negativos: {suma}")
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 68_control_sumar_positivos.py

Salida de referencia

```
Suma de los números no negativos: 35
```

Conceptos clave: break, continue, pass, for y while.

69. Marcador condicional pass

Requerimiento de la actividad: Diseñe un if sin cuerpo provisional.

Archivo: 69_control_if_con_pass.py

```
numero = int(input("Ingrese un número: "))

if numero > 0:
    pass
else:
    print("El número no es positivo.")

print("Validación terminada.")
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 69_control_if_con_pass.py

Datos de ejemplo: 3

Salida de referencia

```
Ingrese un número: Validación terminada.
```

Conceptos clave: break, continue, pass, for y while.

70. Interrupción de emergencia

Requerimiento de la actividad: Simule botón de parada de emergencia con break.

Archivo: 70_control_parada_emergencia.py

```
while True:
    comando = input("Escriba 'continuar' o 'detener': ").strip().lower()
    if comando == "detener":
        print("Parada de emergencia activada.")
        break
    if comando == "continuar":
        print("Sistema en funcionamiento.")
    else:
        print("Comando no reconocido.")
```

Cómo funciona: break termina el ciclo, continue omite una iteración y pass sirve como instrucción vacía.

Cómo ejecutar: python 70_control_parada_emergencia.py

Datos de ejemplo: continuar | detener

Salida de referencia

```
Escriba 'continuar' o 'detener': Sistema en funcionamiento.
Escriba 'continuar' o 'detener': Parada de emergencia activada.
```

Conceptos clave: break, continue, pass, for y while.

71. Ejercicio Integrador 1: Sistema de Autenticación y Control de Intentos

Requerimiento de la actividad: Desarrolle un programa en Python que simule el inicio de sesión de un sistema académico en Visual Studio Code. El usuario dispone máximo de 3 intentos para ingresar la contraseña correcta ("SENA2026"). Si acierta, el sistema muestra "Acceso Concedido" y se detiene inmediatamente mediante control de flujo. Si agota los 3 intentos, el sistema se bloquea e imprime una alerta de seguridad.

Archivo: 71_integrador_sistema_autenticacion.py

```
contrasena_correcta = "SENA2026"
intentos_maximos = 3

for intento in range(1, intentos_maximos + 1):
    contrasena = input(f"Intento {intento} de {intentos_maximos}. Ingrese la contraseña: ")
    if contrasena == contrasena_correcta:
        print("Acceso Concedido")
        break
    print("Contraseña incorrecta.")
else:
    print("Sistema bloqueado: alerta de seguridad.")
```

Cómo funciona: La solución combina entradas, condicionales y ciclos para procesar datos y presentar resultados.

Cómo ejecutar: python 71_integrador_sistema_autenticacion.py

Datos de ejemplo: mal | SENA2026

Salida de referencia

```
Intento 1 de 3. Ingrese la contraseña: Contraseña incorrecta.
Intento 2 de 3. Ingrese la contraseña: Acceso Concedido
```

Conceptos clave: listas, input(), for, condicionales y acumuladores.

72. Ejercicio Integrador 2: Calculadora de Inventario y Estadísticas de Productos

Requerimiento de la actividad: Diseñe un script que procese un lote de 5 productos registrados. Para cada producto se ingresa su precio. El programa debe calcular el costo total del inventario, identificar cuántos productos superan el precio umbral de $50,000, determinar cuál fue el producto más costoso y mostrar un reporte estadístico completo utilizando ciclos, condicionales y operadores de comparación.

Archivo: 72_integrador_estadisticas_inventario.py

```
precios = []

for numero_producto in range(1, 6):
    precio = float(input(f"Ingrese el precio del producto {numero_producto}: "))
    precios.append(precio)

total = 0
productos_sobre_umbral = 0
producto_mas_costoso = precios[0]

for precio in precios:
    total += precio
    if precio > 50_000:
        productos_sobre_umbral += 1
    if precio > producto_mas_costoso:
        producto_mas_costoso = precio

print("\nReporte del inventario")
print(f"Costo total: ${total:,.2f}")
print(f"Productos que superan $50,000: {productos_sobre_umbral}")
numero_producto_mas_costoso = precios.index(producto_mas_costoso) + 1
print(
    f"Producto más costoso: Producto {numero_producto_mas_costoso} "
    f"(${producto_mas_costoso:,.2f})"
)
```

Cómo funciona: La solución combina entradas, condicionales y ciclos para procesar datos y presentar resultados.

Cómo ejecutar: python 72_integrador_estadisticas_inventario.py

Datos de ejemplo: 10000 | 60000 | 75000 | 30000 | 50000

Salida de referencia

```
Ingrese el precio del producto 1: Ingrese el precio del producto 2: Ingrese el precio del producto 3: Ingrese
el precio del producto 4: Ingrese el precio del producto 5:
Reporte del inventario
Costo total: $225,000.00
Productos que superan $50,000: 2
Producto más costoso: Producto 3 ($75,000.00)
```

Conceptos clave: listas, input(), for, condicionales y acumuladores.

73. Lista de compras semanal

Requerimiento de la actividad: Cree una lista con 5 artículos de supermercado y muéstrela en consola.

Archivo: 73_listas_compras_semanal.py

```
compras = ["Arroz", "Leche", "Pan", "Huevos", "Frutas"]

print("Lista de compras semanal:")
for articulo in compras:
    print(f"- {articulo}")
```

Cómo funciona: La solución crea o procesa una lista y utiliza sus elementos o métodos para obtener el resultado solicitado.

Cómo ejecutar: python 73_listas_compras_semanal.py

Salida de referencia

```
Lista de compras semanal:
- Arroz
- Leche
- Pan
- Huevos
- Frutas
```

Conceptos clave: listas, índices, append(), insert(), remove(), sorted() y count().

74. Acceso a notas de aprendices

Requerimiento de la actividad: Dada una lista de 5 notas, imprima la primera y la última nota usando índices negativos.

Archivo: 74_listas_notas_aprendices.py

```
notas = [4.2, 3.8, 4.5, 3.9, 4.8]

print(f"Primera nota: {notas[0]}")
print(f"Última nota: {notas[-1]}")
```

Cómo funciona: La solución crea o procesa una lista y utiliza sus elementos o métodos para obtener el resultado solicitado.

Cómo ejecutar: python 74_listas_notas_aprendices.py

Salida de referencia

```
Primera nota: 4.2
Última nota: 4.8
```

Conceptos clave: listas, índices, append(), insert(), remove(), sorted() y count().

75. Modificar inventario

Requerimiento de la actividad: Modifique el tercer producto de una lista de inventario por "Teclado Mecánico".

Archivo: 75_listas_modificar_inventario.py

```
inventario = ["Monitor", "Mouse", "Teclado", "Audífonos", "Webcam"]
inventario[2] = "Teclado Mecánico"

print(f"Inventario actualizado: {inventario}")
```

Cómo funciona: La solución crea o procesa una lista y utiliza sus elementos o métodos para obtener el resultado solicitado.

Cómo ejecutar: python 75_listas_modificar_inventario.py

Salida de referencia

```
Inventario actualizado: ['Monitor', 'Mouse', 'Teclado Mecánico', 'Audífonos', 'Webcam']
```

Conceptos clave: listas, índices, append(), insert(), remove(), sorted() y count().

76. Registro de nuevos aprendices

Requerimiento de la actividad: Agregue dos nombres nuevos al final de una lista usando append().

Archivo: 76_listas_nuevos_aprendices.py

```
aprendices = ["Ana", "Carlos", "Luisa"]
aprendices.append("Mateo")
aprendices.append("Sofía")

print(f"Lista de aprendices: {aprendices}")
```

Cómo funciona: La solución crea o procesa una lista y utiliza sus elementos o métodos para obtener el resultado solicitado.

Cómo ejecutar: python 76_listas_nuevos_aprendices.py

Salida de referencia

```
Lista de aprendices: ['Ana', 'Carlos', 'Luisa', 'Mateo', 'Sofía']
```

Conceptos clave: listas, índices, append(), insert(), remove(), sorted() y count().

77. Inserción posicional

Requerimiento de la actividad: Inserte la materia "Matemáticas" en la segunda posición de una lista de asignaturas.

Archivo: 77_listas_insercion_posicional.py

```
asignaturas = ["Programación", "Bases de datos", "Redes"]
asignaturas.insert(1, "Matemáticas")

print(f"Asignaturas: {asignaturas}")
```

Cómo funciona: La solución crea o procesa una lista y utiliza sus elementos o métodos para obtener el resultado solicitado.

Cómo ejecutar: python 77_listas_insercion_posicional.py

Salida de referencia

```
Asignaturas: ['Programación', 'Matemáticas', 'Bases de datos', 'Redes']
```

Conceptos clave: listas, índices, append(), insert(), remove(), sorted() y count().

78. Depuración de lista

Requerimiento de la actividad: Elimine un elemento específico por su nombre utilizando remove().

Archivo: 78_listas_eliminar_elemento.py

```
productos = ["Monitor", "Mouse", "Teclado", "Audífonos"]
producto_eliminar = input("Ingrese el nombre del producto que desea eliminar: ")

if producto_eliminar in productos:
    productos.remove(producto_eliminar)
    print(f"Inventario actualizado: {productos}")
else:
    print("El producto no está en la lista.")
```

Cómo funciona: La solución crea o procesa una lista y utiliza sus elementos o métodos para obtener el resultado solicitado.

Cómo ejecutar: python 78_listas_eliminar_elemento.py

Datos de ejemplo: Mouse

Salida de referencia

```
Ingrese el nombre del producto que desea eliminar: Inventario actualizado: ['Monitor', 'Teclado', 'Audífonos']
```

Conceptos clave: listas, índices, append(), insert(), remove(), sorted() y count().

79. Ordenamiento de puntajes

Requerimiento de la actividad: Ordene una lista de puntajes de menor a mayor y luego de mayor a menor.

Archivo: 79_listas_ordenar_puntajes.py

```
puntajes = [78, 95, 62, 88, 71]

puntajes_ascendentes = sorted(puntajes)
puntajes_descendentes = sorted(puntajes, reverse=True)

print(f"De menor a mayor: {puntajes_ascendentes}")
print(f"De mayor a menor: {puntajes_descendentes}")
```

Cómo funciona: La solución crea o procesa una lista y utiliza sus elementos o métodos para obtener el resultado solicitado.

Cómo ejecutar: python 79_listas_ordenar_puntajes.py

Salida de referencia

```
De menor a mayor: [62, 71, 78, 88, 95]
De mayor a menor: [95, 88, 78, 71, 62]
```

Conceptos clave: listas, índices, append(), insert(), remove(), sorted() y count().

80. Conteo de asistencias

Requerimiento de la actividad: Cuente cuántas veces aparece la palabra "Presente" en una lista de asistencia.

Archivo: 80_listas_contar_asistencias.py

```
asistencias = ["Presente", "Ausente", "Presente", "Presente", "Ausente"]
total_presentes = asistencias.count("Presente")

print(f"Cantidad de asistencias: {total_presentes}")
```

Cómo funciona: La solución crea o procesa una lista y utiliza sus elementos o métodos para obtener el resultado solicitado.

Cómo ejecutar: python 80_listas_contar_asistencias.py

Salida de referencia

```
Cantidad de asistencias: 3
```

Conceptos clave: listas, índices, append(), insert(), remove(), sorted() y count().